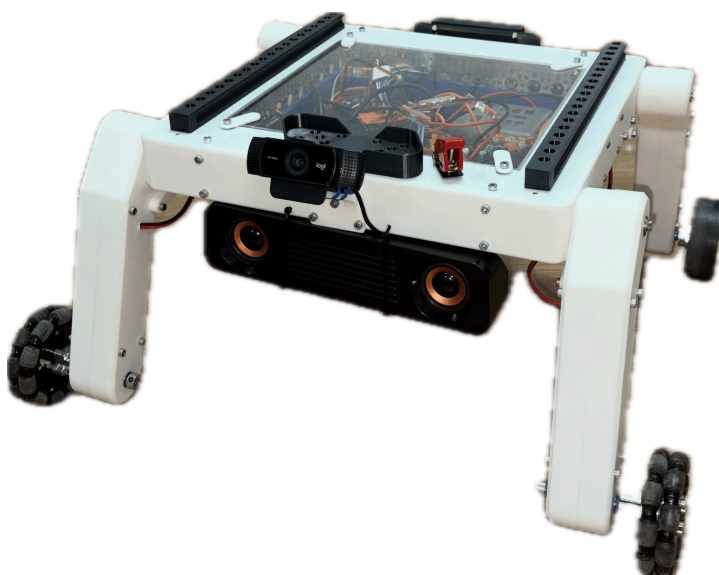


BUDDYBOT

Robot Modular de Asistență Personală

Concurs InfoEducație - Secțiunea Roboți

Mark Aster și Jannik Welzeck



CUPRINS

Capitolul I. Utilitate Practică

- 1.1. Problema Abordată
- 1.2. Soluția Propusă - BuddyBot
- 1.3. Eficiență Comparativ cu Alte Soluții

Capitolul II. Mecanica

- 2.1. Platforma de Bază
- 2.2. Sistemul de Propulsie
- 2.3. Sistemul de Șine și Modularitate
- 2.4. Modulul Braț Robotic (5 axe + claw)
- 2.5. Modulul Coș/Cântar de Cumpărături
- 2.6. Modulul Suport Insta360
- 2.7. Complexitate Mecanică — Motoare și Grade de Libertate
- 2.8. Eficiența în Construcție și Sursa de Energie

Capitolul III. Electronica

- 3.1. Arhitectura Sistemului Electronic
- 3.2. Sursa de Alimentare
- 3.3. Distribuția Semnalelor și a Busurilor
- 3.4. Senzori — Tip, Fenomen Măsurat, Interval, Utilizare
- 3.5. Componente Active — Microprocesoare, Microcontrollere, Circuite Logice
- 3.6. Sistemul de Urmărire (Viziune + BLE Beacon + LiDAR)
- 3.7. Complexitatea Funcțională a Robotului
- 3.8. Cutia de Electronice și Capacul din Plexiglas

Capitolul IV. Software

- 4.1. Arhitectura Software
- 4.2. Vision Tracking — buddy_bot.py
- 4.3. Triangulare BLE — ble_scanner.py
- 4.4. Navigație pe Traseu — navigation.py
- 4.5. Interfața cu Utilizatorul — display.py
- 4.6. Control Vocal al Brațului — voice_module.py
- 4.7. Detectarea Modulelor — usb_module.py / port_manager.py
- 4.8. Scanare Coduri de Bare și Cântărire — barcode_scanner.py
- 4.9. Siguranță — tipover_monitor.py
- 4.10. Feedback Audio
- 4.11. Scripturi de Testare și Diagnostic
- 4.12. Biblioteci și Tehnologii Utilizate
- 4.13. Clasificarea Software-ului Conform Criteriilor de Concurs
- 4.14. Stilul de Programare și Eficiența Codului

Capitolul V. Design Industrial

5.1. Simplitate și Funcționalitate

5.2. Compatibilitate cu Automatizarea Producției

5.3. Dificultăți Întâmpinate

Capitolul VI. Prezentare

6.1. Prezentarea Scrisă

6.2. Prezentarea pe Suport Electronic

Bibliografie

Capitolul I: Utilitate Practică

1.1. Problema Abordată

În viața cotidiană, oamenii se confruntă cu sarcini repetitive și consumatoare de timp: căratul bagajelor sau al uneltelor, filmarea, ridicarea și manipularea obiectelor, monitorizarea cheltuielilor la cumpărături sau simpla nevoie de a avea un ajutor mobil care să urmeze o persoană fără să fie condus manual tot timpul. Soluțiile existente sunt fie prea specializate (roboți industriali fixați pe o singură sarcină, imobili), fie prea limitate (roboți de urmărire simpli, fără posibilitatea de a schimba ce fac).

Problema pe care o abordăm este lipsa unui robot personal accesibil și ușor adaptabil, capabil să ajute în contexte diferite: o hală de producție, o plimbare cu câinele, o drumeție sau cumpărăturile de acasă, fără reconfigurare complexă sau cunoștințe tehnice din partea utilizatorului.

1.2. Soluția Propusă - BuddyBot

BuddyBot este un robot patruped, cu o platformă de bază pe care se pot monta și demonta module funcționale în funcție de sarcina dorită. Platforma de bază integrează urmărirea unei persoane (cameră + beacon-uri BLE), propulsie prin patru roți motorizate independent și control prin display, gamepad sau o pagină web.

Fiecare picior este acționat de un motor GoBilda Yellow Jacket la 312 RPM; cuplul este transmis către roată printr-o pereche de angrenaje conice (bevel gears), care schimbă direcția de rotație cu 90° de la axul vertical al motorului la axul orizontal al roții. În față sunt montate roți omnidirecționale de 72 mm diametru, care reduc frecarea laterală la viraj, iar în spate roți de tracțiune tip rhino, de același diametru, pentru aderență pe teren accidentat.

Modulele proiectate până acum sunt:

- **Braț robotic cu 5 axe și claw interschimbabil**, controlat vocal prin microfonul camerei web, cu cinematică inversă (IK) pentru poziționarea gheare, pentru ridicarea și manipularea obiectelor sau sarcini de tip pick-and-place.
- **Coș de cumpărături cu cântar integrat**, pentru urmărirea greutății și a costului produselor scanate.
- **Suport Insta360**, pentru filmarea la persoana 1 sau urmărire cinematică, fără nicio conexiune electrică.

Orice modul nou, respectând standardul mecanic și electric al șinelor, poate fi adăugat ulterior fără modificări asupra platformei de bază. Modulele se montează pe două șine liniare, dispuse pe lateralele suprafeței superioare a robotului. Fiecare modul are propriul microcontroller ESP32; comunicarea cu Raspberry Pi-ul central se face printr-un singur conector USB de tip mamă, expus prin capacul de acril al cutiei de electronice — un singur modul poate fi conectat electric la un moment dat, deși mai multe pot fi montate mecanic simultan pe șine.

Comparativ cu alte moduri de a rezolva aceleași sarcini — un cărucior tras manual, un angajat dedicat exclusiv cărăușului de unelte, sau o cameră de acțiune ținută în mână — BuddyBot elimină nevoia ca o persoană să opereze continuu dispozitivul: robotul urmărește singur utilizatorul (prin cameră și, când acesta iese din cadru, prin triangulare BLE), cântărește și scanează produsele fără o casă de marcat separată, și schimbă funcția fizică (transport, cumpărături, filmare) prin înlocuirea unui modul, nu prin cumpărarea unui echipament separat pentru fiecare sarcină.

1.3. Eficiență Comparativ cu Alte Soluții

Argumentele pentru care problema este rezolvată mai eficient cu BuddyBot decât prin alte metode:

- **Un singur robot, mai multe sarcini:** modulele interschimbabile (braț, coș/cântar, suport cameră) înlocuiesc mai multe unelte sau dispozitive separate, fiecare cu propriul cost și propria sursă de alimentare.
- **Fără operator continuu:** urmărirea persoanei prin cameră, completată de triangulare BLE atunci când utilizatorul iese din cadru, elimină nevoia de a conduce robotul manual pentru sarcinile de transport/urmărire; controlul manual (gamepad sau pagină web) rămâne disponibil ca alternativă, nu ca singura opțiune.
- **Interfață accesibilă fără echipament suplimentar:** display-ul TFT montat pe robot oferă control local complet; în plus, Raspberry Pi-ul expune propriul access point Wi-Fi cu o interfață web, accesibilă de pe orice telefon sau laptop conectat la acesta, fără o aplicație dedicată.
- **Feedback direct:** un difuzor montat în față, sub cameră, oferă confirmări vocale în limba română pentru fiecare acțiune importantă (pornire motoare, conectare gamepad, conectare modul, avertizare de răsturnare), astfel încât utilizatorul nu trebuie să verifice constant un ecran pentru a ști ce face robotul.
- **Alimentare separată pe module:** modulele cu consum mai mare pot avea sursă de alimentare proprie, astfel încât activarea lor nu afectează tensiunea disponibilă pentru platforma de bază.

Capitolul II: Mecanica

2.1. Platforma de Bază

BuddyBot este construit ca un patruped: o cutie de electronice centrală, ținută la aproximativ 22 cm față de sol de patru picioare motorizate independent. Înălțimea a fost aleasă pentru a păstra o gardă la sol suficientă pentru teren neuniform (iarbă, praguri, pietriș), fără a afecta stabilitatea robotului.

Cutia de electronice găzduiește componentele centrale (Raspberry Pi, drivere de motor, sursa de alimentare, IMU etc.) și este acoperită de un capac din Plexiglas, decupat la dimensiune cu un cutter cu fir cald. Capacul se fixează printr-un mecanism cu balama culisantă, similar cu modul în care se închide capacul modulului AMS 2 de la Bambu Lab, ceea ce permite acces rapid la interior pentru depanare, fără demontarea completă a cutiei. Dimensiunea robotului este de **59x55x39cm**.

Suprafața superioară a cutiei este dedicată sistemului de șine liniare pentru module (secțiunea 2.3); pe partea din spate este montat display-ul TFT de interfațare, iar pe partea din față camera web și difuzorul.

2.2. Sistemul de Propulsie

Spre deosebire de un robot pe șenile sau cu roți montate direct pe șasiu, BuddyBot folosește patru picioare, fiecare cu propriul motor și propria roată la bază. Fiecare motor este un GoBilda Yellow Jacket la 312 RPM, montat vertical în interiorul piciorului; cuplul este transmis către axul roții printr-o pereche de angrenaje conice, care schimbă direcția de rotație cu 90° de la axul motorului la axul roții.

Componentă	Detalii
Motoare de propulsie	4× GoBilda Yellow Jacket, 312 RPM
Transmisie	Angrenaje conice (bevel gears), 90°, motor→roată, câte o pereche per picior
Roți față	2× roți omnidirecționale, Ø 72 mm
Roți spate	2× roți de tracțiune tip rhino, Ø 72 mm
Configurație electrică	Tracțiune diferențială (skid-steer), stânga/dreapta independente
Drivere motoare	2× BTS7960 43A H-Bridge (câte 2 motoare/driver — un driver per parte)
Control	PCA9685 (I ² C) generează PWM-ul pentru cele două drivere BTS7960, comandat de Raspberry Pi 3B+

Roțile omnidirecționale din față reduc frecarea laterală la viraj, iar roțile de tracțiune din spate oferă aderență pe teren accidentat (iarbă, pietriș, pante ușoare); combinația celor două tipuri de roți este alegerea de proiectare centrală a sistemului de propulsie.

Motoarele GoBilda Yellow Jacket au fost alese pentru cuplul lor la o viteză de rotație moderată, iar driverele BTS7960 suportă curenți de până la 43A, cu protecție termică și la supracurent. Planul inițial prevedea un driver per motor (patru drivere), dar mai multe drivere au ars din cauza unor greșeli de cablare; soluția finală, mai simplă, folosește doar două drivere, câte unul per parte, fiecare controlând în paralel cele două motoare de pe partea respectivă.

2.3. Sistemul de Șine și Modularitate

Pe suprafața superioară a cutiei de electronice sunt montate două șine liniare, dispuse pe cele două laturi ale robotului, pe care modulele se glisează și se fixează. Fiecare modul se conectează electric printr-un singur conector USB de tip mamă, expus prin capacul de acril. Un cablu USB tată-mamă din interior face legătura între acest conector expus și un port USB al Raspberry Pi-ului (secțiunea 3.4).

La un moment dat poate fi conectat electric un singur modul, însă mai multe module pot fi montate mecanic simultan pe cele două șine, pregătite pentru a fi comutate fără scule atunci când utilizatorul dorește să schimbe funcția robotului. Sina/profil folosit este unul printat 3D, iar mecanismul de fixare este unul simplu, prin intermediul a doua betisoare tot 3D printate, pe post de barieră.

2.4. Modulul Braț Robotic (5 Axe + Claw)

Brațul robotic oferă 5 grade de libertate plus o gheară (claw) interschimbabilă, acoperind un spațiu de lucru adecvat pentru manipulare la nivel de masă sau de podea. Poziționarea gherei se face prin cinematică inversă (IK): utilizatorul dă o comandă de nivel înalt, iar firmware-ul modulului calculează unghiurile necesare pentru fiecare servo.

Element	Detalii
Axa 1 (rotație bază)	Servomotor
Axa 2 (umăr)	Servomotor
Axa 3 (cot)	Servomotor
Axa 4 (încheietură/rotație)	Servomotor
Axa 5 (claw/gripper)	Servomotor
Microcontroller modul	ESP32-C3 Super Mini
Comunicare cu RPi	Serial prin USB, prin conectorul expus pe șine
Control	Comenzi vocale ("open", "close", "wave", "grab", "throw" etc.), recunoscute local prin microfonul camerei web

Claw-ul de la capătul brațului are un cuplaj rapid interschimbabil, care permite montarea altor atașamente. Modulul raportează continuu, prin serial, unghiul fiecărui servo (bază, primul segment, al doilea segment, al treilea segment, claw, unghi claw), date afișate pe display-ul TFT al robotului.

2.5. Modulul Coș de Cumpărături

Modulul coș de cumpărături este o structură ușoară montată pe șinele platformei. Integrează un sistem de cântărire și folosește camera web a robotului pentru scanarea codurilor de bare ale produselor.

Element	Detalii
Senzor greutate	Load Cell 20 kg
Convertor ADC	HX711, 24-bit
Scanner coduri de bare	Integrat în camera web (OpenCV + pyzbar)
Microcontroller modul	ESP32-C3 Super Mini

Load cell-ul de 20 kg este montat în baza coșului și măsoară greutatea totală a produselor adăugate. HX711 convertește semnalul analogic diferențial al load cell-ului în date digitale de 24 biți, cu o rezoluție de sub 1 gram. ESP32-ul modulului transmite valoarea ("Weight: ...") prin serial USB către Raspberry Pi, care o afișează pe display alături de coșul curent și prețul total.

2.6. Modulul Suport Insta360

Suportul pentru camera Insta360 este cel mai simplu modul din punct de vedere mecanic: o structură rigidă montată pe șine, care ține camera stabilizată la înălțimea dorită. Modulul nu are microcontroller propriu și nu folosește conectorul USB expus, deoarece camera Insta360 nu are nevoie de o legătură de date sau de alimentare de la robot.

2.7. Complexitate Mecanică — Motoare și Grade de Libertate

Numărul de motoare/servomotoare depinde de configurația robotului la un moment dat (adică ce modul este montat):

Subansamblu	Nr. motoare	Grade de libertate
Platforma de bază (4 picioare)	4 (motoare DC, câte unul per picior)	2 DoF de control efectiv (înainte/înapoi și viraj), obținute prin diferența de viteză stânga/dreapta; fiecare roată se rotește continuu, fără o articulație suplimentară pe picior.
Modul braț robotic	6 (5 axe în serie + 1 pentru claw)	5 DoF în serie pentru poziționarea brațului, plus 1 DoF pentru deschiderea/închiderea gheare — 6 DoF în total pe modul.
Modul coș/cântar	0 (fără actuatori — doar senzor)	0 — structură fixă.
Modul suport Insta360	0	0 — structură fixă; orientarea camerei este dată de camera Insta360 însăși, nu de o articulație a suportului.

Robotul complet (platformă + braț montat) însumează 10 motoare/servomotoare (4 de propulsie + 6 pe brațul robotic) și 7 grade de libertate de control distincte (2 pentru deplasare + 5 pentru braț, plus deschiderea gherei).

2.8. Eficiența în Construcție și Sursa de Energie

Consumul de energie este redus prin decizii atât mecanice, cât și software:

- Motoarele de propulsie sunt oprite complet (nu doar încetinite) atunci când persoana urmărită este suficient de aproape sau când nu se detectează nicio țintă, în loc să rămână active la o viteză minimă în mod continuu.
- Cele două drivere BTS7960 (câte unul per parte, în loc de patru — câte unul per motor) reduc numărul de componente care consumă curent de mentenanță (standby) în paralel.
- PCA9685 generează semnalele PWM pentru motoare pe cale hardware dedicată, fără ca Raspberry Pi-ul să mențină un thread separat, cu consum de CPU (deci indirect, de energie), pentru fiecare canal de motor.
- Pe partea de calcul, interfața de pe display se redesenează doar când starea afișată se schimbă efectiv, nu la o rată fixă — reduce sarcina pe CPU și, prin extensie, consumul Raspberry Pi-ului.

Referitor la sursa de energie: BuddyBot nu are o sursă de energie regenerabilă. Sursa principală este un acumulator LiPo 3S 2200 mAh.

În privința timpului de acumulare versus timpul de funcționare: un acumulator LiPo 3S 2200 mAh se încarcă, la un curent uzual de încărcare de 1C, în aproximativ o oră, folosind un încărcător de tip balance charger dedicat acumulatorilor LiPo. Timpul de funcționare efectiv depinde puternic de modulul folosit și de cât de mult sunt solicitate motoarele de propulsie și brațul robotic; nu am realizat încă o măsurătoare formală a autonomiei robotului sub sarcină tipică.

Curenții măsurați pe robot sunt:

Raspberry Pi în repaus (idle) — 0.4–0.6 A; motoare la viteză maximă susținută — 1.8 A; vârf la schimbarea agresivă a direcției motoarelor — 5 A; sistem "oprit" — 0.22 A. Acumulatorul folosit este un LiPo 3S de 2200 mAh (2.2 Ah).

Formula folosită este simplă: $\text{timp de funcționare (h)} = \text{Capacitate (Ah)} / \text{Curent (A)}$. Pentru un LiPo, este recomandat să nu se consume toată capacitatea nominală (pentru a proteja celulele și a evita căderea bruscă de tensiune la final) — considerăm o capacitate utilizabilă practică de aproximativ 80% din cea nominală, adică 1.76 Ah.

- Cel mai bun caz (robot în repaus, fără mișcare a motoarelor), 0.4–0.6 A: $2.2 \text{ Ah} / 0.4\text{--}0.6 \text{ A} \approx \mathbf{3 \text{ h } 40 \text{ min} - 5 \text{ h } 30 \text{ min}}$ teoretic (nominal), respectiv $\mathbf{2 \text{ h } 56 \text{ min} - 4 \text{ h } 24 \text{ min}}$ folosind capacitatea utilizabilă de 1.76 Ah.
- Cel mai rău caz susținut (motoare la viteză maximă continuu), 1.8 A: $2.2 \text{ Ah} / 1.8 \text{ A} \approx \mathbf{1 \text{ h } 13 \text{ min}}$ teoretic, respectiv $\mathbf{59 \text{ min}}$ folosind capacitatea utilizabilă.
- Vârful de 5 A (schimbare agresivă de direcție) apare doar în impulsuri de ordinul sutelor de milisecunde/câtorva secunde, nu ca regim continuu — dacă s-ar menține tot timpul la 5 A, autonomia teoretică ar fi doar $2.2 \text{ Ah} / 5 \text{ A} \approx \mathbf{26 \text{ min}}$, dar acest număr nu reprezintă o utilizare reală, ci doar o limită absolută inferioară.
- Estimare medie, realistă: într-o utilizare tipică (deplasare/urmărire alternând cu perioade de repaus și ocazional vârfuri scurte la schimbarea direcției), estimăm un curent mediu de aproximativ 1.0–1.2 A. Aceasta dă o autonomie de $\mathbf{1 \text{ h } 45 \text{ min} - 2 \text{ h } 12 \text{ min}}$ (folosind capacitatea nominală de 2.2 Ah), respectiv $\mathbf{1 \text{ h } 24 \text{ min} - 1 \text{ h } 45 \text{ min}}$ folosind capacitatea utilizabilă de 1.76 Ah.

Capitolul III: Electronica

3.1. Arhitectura Sistemului Electronic

Arhitectura electronică a BuddyBot este ierarhică și distribuită. Raspberry Pi 3B+ este microprocesorul central, coordonând toți ceilalți microcontrollere și periferice. ESP32-urile modulelor și ale beacon-urilor BLE sunt noduri satelit specializate, care comunică cu Raspberry Pi prin USB serial.

Lanțul general al arhitecturii, de la sursa de energie până la periferice, este:

- Acumulator LiPo 3S 2200 mAh (Soaring) — sursa de putere a platformei.
- Siguranță de tip breaker 50A — și întrerupător principal al bateriei (secțiunea 3.2).
- Cutie de siguranțe auto (6 poziții) — protejează fiecare subsistem individual.
- Coborâtor de tensiune (buck converter) — reglează tensiunea la 5.2V/5A pentru electronica de semnal mic.
- Raspberry Pi 3B+ — coordonator central (urmărire vizuală, agregare BLE, control motoare, server web, interfață display).
- PCA9685 — controlează motoarele prin 3 fire, folosind protocolul I²C.
- 2× BTS7960 — drivere de motor, câte unul per parte a robotului.
- 2× ESP32-C3 (beacon-scanner, montate pe robot, stânga/dreapta) — scanare RSSI BLE, conectate la RPi prin USB serial.
- 1× ESP32 (purtat de utilizator) — beacon BLE emițător.
- 1× ESP32-C3 pe modulul braț — control servomotoare + calcul IK.
- 1× ESP32-C3 pe modulul coș — citire HX711, transmisie greutate.
- Senzor IMU 9 axe ICM-20948 — orientare pentru navigație și detecția răsturnării.
- Display TFT SPI (montat pe spatele robotului) — interfață utilizator la bordul robotului.
- LiDAR rotativ (montat sub cutia de electronice) — prezent electric/mecanic, dar nefuncțional (secțiunea 3.6).
- Difuzor Bluetooth (montat în față, sub cameră) — feedback audio.

3.2. Sursa de Alimentare

Alimentarea BuddyBot este structurată pe mai multe niveluri de protecție, rezultate din problemele întâmpinate la versiunile anterioare ale robotului:

- **Sursă principală:** acumulator LiPo 3S 2200 mAh, marca Soaring, alimentează întregul robot.
- **Siguranță breaker 50A:** conectată direct la baterie, este întrerupătorul principal al întregului sistem, deconectând bateria în caz de scurtcircuit sau supracurent. Din cauza managementului de cabluri din cutia de electronice, acest breaker este momentan greu accesibil fizic — de îmbunătățit într-o versiune viitoare.
- **Cutie de siguranțe (fusebox) cu 6 poziții,** similară celor din industria auto. Ramurile alimentate sunt: două siguranțe de 20A, câte una pentru fiecare driver de motor BTS7960, și o siguranță de 5A pentru ramura coborâtorului de tensiune.
- **Coborâtor de tensiune (buck converter):** setat la o ieșire de 5.2V, curent maxim 5A. Ieșirea trece printr-un întrerupător de tip "aviație", montat pe partea superioară a cutiei, protejat de un capac.
- **De la întrerupător la Raspberry Pi:** cei 5.2V ajung într-un conector WAGO; de pe partea cealaltă pornește un cablu Dupont de aproximativ 1 cm, conectat direct la pinul de 5V și la GND al Raspberry Pi, pentru a minimiza căderea de tensiune pe acest ultim segment.

- **Baterii separate pe module:** unele module (de exemplu brațul robotic sau coșul) pot avea surse de alimentare proprii, astfel încât activarea lor nu afectează tensiunea disponibilă pentru Raspberry Pi.

Motivul acestei arhitecturi pe mai multe niveluri este legat direct de cea mai mare problemă întâlnită la o versiune anterioară a robotului: foloseam cabluri Dupont lungi pentru alimentare, care pierdeau tensiune pe traseu din cauza lungimii și a rezistenței lor, cauzând resetări (undervoltage) ale Raspberry Pi-ului chiar la sarcini de curent moderate. Soluția a fost reducerea lungimii oricărui cablu Dupont din calea de 5V (înlocuind legăturile lungi cu conectori WAGO) și adăugarea unui condensator pe pinii 5V/GND ai Raspberry Pi, ca măsură suplimentară de stabilizare locală a tensiunii.

3.3. Distribuția Semnalelor și a Busurilor

Pentru a evita un cablaj dezorganizat — o problemă a versiunilor anterioare — busurile comune (I²C, alimentare 3.3V, masă) sunt distribuite prin conectori WAGO cu deschideri multiple (6 deschideri, câte 3 pe fiecare parte a conectorului):

- Bus I²C: un WAGO cu 6 deschideri distribuie SDA și SCL către toate perifericele I²C (PCA9685, IMU ICM-20948 etc.).
- Bus 3.3V: un WAGO cu 6 deschideri distribuie alimentarea de 3.3V, cu masă comună pentru toate perifericele conectate.
- PCA9685: conectat pe busul I²C de mai sus, comandă cele două drivere BTS7960 folosind doar 3 fire (SDA, SCL, alimentare/masă), în loc de un PWM separat de pe GPIO pentru fiecare motor.
- Senzor IMU ICM-20948: conectat pe același bus I²C/3.3V. Este montat cu susul în jos în cutia de electronice, cu bandă izolatoare deasupra, pentru a preveni scurtcircuiturile cu alte componente din jur. Orientarea sa este compensată direct în software (secțiunea 4.4).
- Display SPI 3.1": conectat direct pe busul SPI hardware al Raspberry Pi, fără multiplexor, întrucât este singurul periferic SPI din sistem.
- Hub USB: conectat la un port USB al Raspberry Pi, găzduiește cele două ESP32-C3 folosite ca beacon-scanner BLE (montate pe lateralele robotului) și alimentarea difuzorului.
- Celelalte porturi USB ale Raspberry Pi: un adaptor Bluetooth USB dedicat (chipset-ul Bluetooth intern al RPi3B+ având o latență prea mare pentru comenzi gamepad-ului) și cablul USB tată-mamă care iese prin capacul de acril, spre conectorul expus al modulelor (secțiunea 2.3).
- Ieșirea audio a difuzorului este conectată la mufa jack 3.5mm a Raspberry Pi-ului.
- LiDAR rotativ: conectat prin pini seriali GPIO (UART) direct la Raspberry Pi, alimentat la 5V.

Toate modulele (braț, coș) au carcase sau suporturi proprii, imprimate 3D, care izolează fizic plăcile electronice și conexiunile expuse, reducând riscul de scurtcircuit.

3.4. Senzori — Tip, Fenomen Măsurat, Interval, Utilizare

Fiecare senzor de mai jos are un rol direct în rezolvarea unei sarcini a robotului; nu am adăugat senzori suplimentari fără o utilizare clară în funcționarea robotului.

Senzor	Fenomen măsurat	Interval de intrare	Utilizare
Cameră web (webcam)	Lumină vizibilă (imagine RGB)	640×480 px, până la 30 cadre/s	Detecția și urmărirea persoanei (computer vision), scanarea codurilor de bare, captura audio pentru comenzi vocale (prin microfonul integrat).
IMU ICM-20948 — accelerometru	Accelerație liniară pe 3 axe	±2 g (interval implicit)	Calculul roll/pitch al robotului, folosit atât pentru heading în

Senzor	Fenomen măsurat	Interval de intrare	Utilizare
			navigație, cât și pentru detecția răsturnării.
IMU ICM-20948 — giroscop	Viteză unghiulară pe 3 axe	$\pm 250^\circ/\text{s}$ (interval implicit)	Fuziune cu accelerometrul (filtru Madgwick) pentru o estimare stabilă a orientării robotului în timpul deplasării.
IMU ICM-20948 — magnetometru	Câmp magnetic pe 3 axe	$\pm 4900 \mu\text{T}$ (interval implicit)	Calculul unui heading de tip busolă, calibrat pentru a elimina distorsiunea magnetică introdusă de motoare și șasiu.
Load Cell 20 kg + HX711	Forță (greutate), prin deformarea unei mărci tensometrice	0–20 kg, rezoluție sub 1 g (ADC 24 biți)	Cântărirea produselor din coșul de cumpărături, afișată continuu pe display și în interfața web.
2× ESP32-C3 BLE Scanner	Puterea semnalului radio recepționat (RSSI)	aproximativ -30 până la -95 dBm	Estimarea direcției utilizatorului atunci când camera nu mai detectează persoana, prin compararea RSSI stânga/dreapta.
LiDAR rotativ (M1C1)	Timp de zbor al unui semnal optic, pentru distanța până la obstacole	proiectat pentru aprox. 0.15–8 m, 360°	Destinat evitării obstacolelor; modulul montat pe robot a ajuns defect din fabricație și transmite date incoerente, deci nu este folosit efectiv în deciziile robotului (secțiunea 3.6).

3.5. Componente Active — Microprocesoare, Microcontrollere, Circuite Logice

În ordine descrescătoare a complexității, fără a include componente electronice pasive (rezistențe, condensatoare, conectori):

Microprocesoare

Componentă	Justificare
Raspberry Pi 3B+ (CPU Cortex-A53 1.4GHz quad-core, 1GB RAM)	Este singura componentă din sistem capabilă să ruleze simultan un sistem de operare complet (Linux), procesare video cu o rețea neuronală, servere web și mai multe fire de execuție paralele — sarcini care depășesc cu mult capacitatea unui microcontroller. Este ales ca nod central tocmai pentru că trebuie să coordoneze toate celelalte componente.

Microcontrollere

Componentă	Justificare
ESP32-C3 Super Mini (×4)	Ales pentru modulele satelit (braț, coș, cei doi beacon-scanner BLE) datorită dimensiunii reduse, consumului mic și suportului nativ pentru Bluetooth Low Energy — util direct pentru rolul de scanare/emisie BLE, fără hardware suplimentar.

Componentă	Justificare
ESP32 (×1, purtat de utilizator)	Emite continuu pachete BLE cu putere fixă, pentru a fi folosit ca beacon de triangulare; nu are nevoie de resursele mai mari ale unui ESP32-C3 dedicat unui modul, un ESP32 standard fiind suficient pentru acest rol simplu.

Circuite logice

Componentă	Justificare
PCA9685 (controller PWM pe 16 canale, I ² C)	Mută generarea semnalelor PWM ale motoarelor din software (pe GPIO-ul Raspberry Pi) într-un circuit dedicat, eliberând CPU-ul și reducând numărul de fire de cablaj necesare între Raspberry Pi și drivere de la unul per canal PWM la doar 3 (SDA, SCL, alimentare).
HX711 (convertor analog-digital 24 biți, dedicat mărcilor tensometrice)	Convertește semnalul diferențial de câțiva milivolți al load cell-ului într-o valoare digitală cu o rezoluție suficientă pentru cântărirea produselor cu o precizie de ordinul gramului, fără a necesita un ADC de uz general și amplificare separată.

Componente electronice active

Componentă	Justificare
2× BTS7960 43A H-Bridge	Driverile de motor trebuie să suporte curentul de pornire al celor 4 motoare GoBilda; BTS7960 oferă protecție termică și la supracurent, redusă la doar două unități (una per parte) după ce configurația inițială cu patru drivere a dus la mai multe defecțiuni prin cablare greșită.
Adaptor Bluetooth USB extern	Chipset-ul Bluetooth intern al Raspberry Pi 3B+ are o latență prea mare pentru citirea promptă a gamepad-ului PS5; un adaptor extern, cu propriul controller Bluetooth, elimină acest gât de sticlă.

3.6. Sistemul de Urmărire (Viziune + BLE Beacon + LiDAR)

Sistemul de urmărire a utilizatorului combină, în principiu, două metode:

- **Computer vision (metoda principală):** camera web furnizează un flux video procesat cu OpenCV; un model MobileNet SSD detectează persoana, iar robotul urmărește centrul acesteia în cadru (detalii software în secțiunea 4.2).
- **Triangulare BLE (metoda de recuperare):** atunci când persoana iese din cadrul camerei, cele două ESP32-C3 montate pe lateralele robotului scanează RSSI-ul beacon-ului purtat de utilizator și estimează direcția acestuia, rotind robotul până la recaptarea vizuală a țintei (detalii software în secțiunea 4.3).

Exista intenția de a folosi și LiDAR-ul rotativ, pentru evitarea obstacolelor în timpul urmăririi, însă modulul montat sub cutia de electronice a ajuns defect din fabricație (transmite date incoerente prin portul serial) și, în consecință, nu este folosit în deciziile robotului. Rămâne montat fizic pe robot, pregătit pentru o eventuală înlocuire a senzorului.

3.7. Complexitatea Funcțională a Robotului

BuddyBot funcționează în trei regimuri distincte, pe care le raportăm explicit față de categoriile din criteriile de jurizare, de la cel mai independent la cel mai controlat:

- **Following Mode (urmărire) — cel mai apropiat de un robot autonom în mediu necunoscut pentru sarcina de urmărire:** robotul nu primește o hartă sau o poziție fixă a utilizatorului; acesta poate fi oriunde în raza camerei sau a beacon-urilor BLE, iar robotul reacționează live la poziția lui, fără un traseu pre-stabilit. Nu realizează însă o hartă completă a mediului și nu evită automat obstacole necunoscute, întrucât LiDAR-ul montat este nefuncțional — de aceea nu îl încadrăm ca autonomie completă, ci ca cea mai apropiată treaptă disponibilă în stadiul actual.
- **Path Navigation (navigație pe traseu) — robot pre-programat, cu date de intrare variabile:** traseul este definit de utilizator ca o listă de pași (direcție/durată/viteză), editabilă dintr-o pagină web, iar robotul execută traseul corectând live abaterea de la direcția țintă folosind IMU-ul — se încadrează la categoria de robot pre-programat cu date de intrare variabile, conform unor specificații date de utilizator.
- **Control manual (gamepad sau pagină web) — robot telecomandat:** utilizatorul controlează direct viteza și direcția robotului, fie cu gamepad-ul PS5 conectat prin Bluetooth, fie din sliderile paginii web; această opțiune rămâne disponibilă permanent, ca alternativă la celelalte două regimuri.

3.8. Cutia de Electronice și Capacul din Plexiglas

Toate componentele descrise mai sus sunt găzduite în cutia de electronice centrală, acoperită de un capac din Plexiglas decupat la dimensiune cu un cutter cu fir cald și fixat printr-un mecanism cu balama culisantă, asemănător modului de închidere al modulului AMS 2 de la Bambu Lab — permite deschiderea rapidă a cutiei pentru depanare, fără scule și fără a compromite protecția componentelor din interior.

Capitolul IV: Software

4.1. Arhitectura Software

Software-ul BuddyBot rulează pe Raspberry Pi OS și este structurat ca un sistem multi-proces, multi-thread, scris în Python. Raspberry Pi-ul creează propriul access point Wi-Fi, la care se pot conecta atât dispozitivele utilizatorului (telefon/laptop, pentru interfața web), cât și alte periferice de rețea, dacă e cazul.

Procesele independente comunică prin fișiere JSON partajate în /tmp — de exemplu, buddy_bot.py scrie starea urmăririi (status, FPS, direcție, țintă) în buddybot_status.json, iar display.py citește acest fișier pentru a-l reda pe ecranul TFT, fără ca cele două procese să fie cuplate direct. Aceași abordare este folosită pentru starea navigației (nav_status.json / nav_cmd.json), pentru comenzile de control manual primite prin browser (buddybot_manual.json) și pentru starea modulelor USB conectate.

Procesul central este display.py, care pornește și oprește celelalte procese (buddy_bot.py, barcode_scanner.py, navigation.py) ca subprocese, la cerere, în funcție de ecranul/funcția selectată de utilizator din meniu sau din pagina web.

Principalele scripturi Python care alcătuiesc sistemul sunt:

Script	Rol
display.py	Proces central. Randare interfață pe display-ul TFT SPI, citire gamepad Bluetooth, server web (Flask) pentru oglindirea display-ului și control de la distanță, pornirea/oprirea celorlalte scripturi.
buddy_bot.py	Urmărire vizuală: detecție persoană, calcul direcție, comenzi motor prin PCA9685.
navigation.py	Deplasare pe un traseu predefinit, folosind IMU-ul (heading) pentru a menține direcția.
ble_scanner.py	Citire RSSI de la cele două ESP32-C3 beacon-scanner, estimare direcție a utilizatorului prin triangulare.
usb_module.py	Detectarea și citirea modulului conectat prin portul USB expus (coș/cântar sau braț robotic).
voice_module.py	Recunoaștere vocală offline (comenzi pentru brațul robotic), folosind microfonul camerei web.
barcode_scanner.py	Scanare coduri de bare din fluxul video, gestionare coș de cumpărături și total de plată.
tipover_monitor.py	Monitorizare continuă a înclinării robotului; alertă vocală în caz de răsturnare.
port_manager.py	Registru comun de porturi seriale, pentru a preveni deschiderea aceluiași port de către două scripturi simultan.

4.2. Vision Tracking — buddy_bot.py

Scriptul buddy_bot.py deschide fluxul video al camerei web la rezoluția 640×480 și îl procesează într-un thread dedicat de captură, separat de bucla principală de control. Camera este detectată automat, testând în ordine /dev/video1, /dev/video2, /dev/video0.

Detecția persoanei se face cu un model MobileNet SSD, încărcat în format Caffe și rulat cu modulul DNN al OpenCV. Fiecare cadru este redimensionat la 160×120 pixeli înainte de inferență, pentru a reduce timpul de procesare pe hardware-ul limitat al Raspberry Pi 3B+; sunt păstrate doar detecțiile de clasă "persoană" cu

un scor de încredere de minimum 0.35. Inferența rulează într-un thread separat, la o rată fixă de 24 Hz, independent de rata camerei — bucla principală de control motor rulează la rata completă a camerei, refolosind ultima detecție validă.

Pentru urmărirea aceleiași persoane (nu a oricărei persoane apărute în cadru), sistemul folosește un scor bazat pe IoU (Intersection over Union) combinat cu un prag de săritură maximă de 90 de pixeli — dacă o nouă detecție este prea departe de ținta curentă și nu se suprapune suficient cu aceasta, ținta curentă este păstrată. Dacă persoana dispare din cadru, ținta este păstrată în memorie pentru o scurtă perioadă înainte de a fi considerată pierdută, moment în care se activează triangularea BLE.

Comenzile de mișcare sunt calculate în funcție de poziția orizontală a centrului persoanei detectate față de centrul imaginii, cu o zonă moartă pentru a evita oscilații mici, o viteză de rotație proporțională cu abaterea și o viteză de înaintare care scade pe măsură ce persoana ocupă un procent tot mai mare din cadru; robotul se oprește complet atunci când persoana este suficient de aproape, în loc să continue să înainteze la o viteză minimă.

Scriptul suportă și control manual, cu prioritate față de urmărirea automată: fie prin gamepad-ul PS5 DualSense (citit direct prin evdev), fie prin sliderile unei pagini web dedicate (secțiunea 4.5) — ambele surse scriu comenzile în același format, iar cea mai recentă are prioritate.

Starea curentă (status, FPS, direcție/putere motor, țintă detectată, gamepad conectat) este scrisă la 10 Hz într-un fișier JSON, citit de display.py pentru afișare pe ecranul TFT, iar fluxul video adnotat este disponibil ca stream separat, accesibil dintr-un browser.

4.3. Triangulare BLE — ble_scanner.py

Cei doi ESP32-C3 montați pe lateralele robotului transmit continuu, prin serial USB, valori RSSI recepționate de la beacon-ul BLE purtat de utilizator. Scriptul ble_scanner.py identifică automat porturile seriale corecte (căutând printre /dev/ttyUSB* și /dev/ttyACM*, filtrând după conținutul liniilor primite, pentru a distinge beacon-urile BLE de alte periferice seriale, precum modulele braț/coș), folosind un registru comun (port_manager.py) pentru a evita ca două scripturi să deschidă simultan același port.

Semnalul RSSI este filtrat printr-o medie mobilă exponențială ($\alpha = 0.25$), pentru a reduce zgomotul semnalului radio. Din valorile RSSI stânga/dreapta și distanța fixă cunoscută dintre cei doi senzori se calculează poziția aproximativă a beacon-ului față de robot, afișată ca radar 2D pe display (secțiunea 4.5).

Când camera pierde ținta, buddy_bot.py comută pe acest mecanism: dacă diferența RSSI stânga-dreapta depășește un prag, robotul se rotește către partea cu semnal mai puternic; dacă diferența este sub prag, se consideră că utilizatorul este aproximativ în față și robotul înaintază încet, până la recaptarea vizuală a țintei.

4.4. Navigație pe Traseu — navigation.py

Scriptul navigation.py permite robotului să parcurgă un traseu predefinit, format din pași cu o direcție cardinală (N, NE, E, SE, S, SW, W, NW) sau un unghi explicit, o durată și o viteză, editabil dintr-o pagină web dedicată și salvat în fișierul nav_path.json.

Orientarea robotului (heading) se calculează folosind senzorul IMU ICM-20948: giroscopul este calibrat static la pornire (400 de eșantioane, robot nemișcat), iar datele accelerometru+giroscop sunt fuzionate printr-un filtru Madgwick (biblioteca ahrs), care produce o estimare de roll/pitch. Din roll/pitch și din datele brute ale magnetometrului se calculează un heading busolă compensat la înclinare — folosit în locul unghiului yaw produs direct de filtru, care este doar o integrare a giroscopului și derivează în timp, fără o referință absolută.

Deoarece motoarele, cablajele și șasiul metalic distorsionează câmpul magnetic local, există o rutină separată de calibrare a magnetometrului: robotul se rotește pe loc câteva secunde, se înregistrează

minimul și maximul citirilor magnetometrului, iar centrul acestui interval este salvat ca offset în `mag_calibration.json`, scăzut din citirile brute în funcționarea normală.

IMU-ul este montat cu susul în jos în cutia de electronice (secțiunea 3.3); semnele brute ale axelor accelerometrului/girosopului sunt corectate direct în cod (inversare de semn pe axele Y și Z), astfel încât roll/pitch/heading calculate să corespundă orientării fizice a robotului.

Fiecare pas al traseului are două faze: robotul se rotește pe loc (proporțional cu eroarea de heading) până ajunge în toleranță de 2° față de direcția țintă, apoi înaintează pentru durata specificată, aplicând o corecție proporțională mică asupra vitezelor stânga/dreapta pentru a compensa deriva de heading. Traseul poate fi întrerupt sau sărit pas cu pas din interfața de pe display.

4.5. Interfața cu Utilizatorul — `display.py`

Display-ul TFT SPI (480×320 px), montat pe partea din spate a robotului, este controlat direct din `display.py`: interfața este randată în memorie cu Pillow, iar cadrul rezultat este trimis pe magistrala SPI în bucăți de 4096 octeți, respectând limita de transfer a driverului `spidev` pe Raspberry Pi.

`display.py` este structurat pe mai multe fire de execuție separate: citirea gamepad-ului (evdev), calculul statisticilor de sistem (CPU%, temperatură, RAM, tensiune, frecvență, uptime — la fiecare 1.5s), randarea ecranului curent și serverul web (Flask). Fiecare ecran al interfeței își redesenează conținutul doar atunci când starea relevantă s-a schimbat (comparând un indicator ieftin al stării curente cu cel anterior), cu limite proprii de rată pentru ecranele cu actualizare continuă (radarul BLE la 8 Hz, scannerul de coduri de bare la 6 Hz, ceasul din System Overview o dată pe secundă).

Meniul principal, navigabil cu gamepad-ul PS5 DualSense (D-pad/stick stânga pentru navigare, X pentru selecție, O pentru înapoi, Triunghi pentru lansarea/oprirea unui script), oferă următoarele ecrane:

- System Overview — CPU%, temperatură, RAM, tensiune de core, frecvență procesor, uptime.
- Following Mode — starea urmăririi (`buddy_bot.py`): țintă detectată/pierdută, FPS al detectorului, direcție și putere motoare, stare gamepad.
- BLE Radar — vizualizare 2D a poziției estimate a utilizatorului, cu urmă animată și inele de distanță.
- Barcode Scanner — status scanner, ultimul produs scanat cu preț, coșul curent și totalul de plată, plus o previzualizare live a camerei.
- Path Navigation — status `navigation.py`: heading țintă vs. heading curent, progresul pasului activ.
- USB Module — datele modulului conectat, actualizate continuu, (greutatea coșului cu bară de progres 0–20 kg, sau unghiurile celor 5 servouri + claw ale brațului).
- Reboot — repornire controlată a sistemului.

Conținutul display-ului fizic este oglindit printr-un server Flask (port 8081), sub forma unui stream MJPEG, alături de o pagină web care reproduce toate comenzile gamepad-ului (Up/Down/Select/Back/Launch-Stop) ca butoane, permițând controlul robotului direct din browser, fără gamepad. Din aceeași interfață web sunt accesibile un editor grafic al traseului de navigație (adaugă/șterge pași, direcție/durată/viteză) și o pagină de control manual cu două slidere virtuale (înainte/înapoi și viraj stânga/dreapta), utilizabile cu mouse-ul/degetul sau cu tastatura (WASD/săgeți).

4.6. Control Vocal al Brațului — `voice_module.py`

Modulul braț robotic poate fi controlat vocal, prin microfonul integrat al camerei web. Recunoașterea vocală rulează local (fără conexiune la internet), folosind biblioteca Vosk (recunoaștere vocală bazată pe Kaldi) cu un model de limbaj restrâns la un set fix de comenzi ale brațului ("straight", "zero", "open", "close", "wave", "yes", "no", "rise", "lower", "dance", "grab", "throw"), ceea ce crește acuratețea recunoașterii pe un hardware modest precum Raspberry Pi 3B+.

Odată recunoscută o comandă, aceasta este trimisă modulului braț prin `usb_module.py` (secțiunea 4.7), care o transmite pe portul serial al ESP32-ului brațului, unde firmware-ul modulului o traduce într-o secvență de mișcare, calculată prin cinematică inversă pentru pozițiile gherei.

4.7. Detectarea Modulelor — `usb_module.py` / `port_manager.py`

Deoarece un singur modul poate fi conectat electric la un moment dat (prin conectorul USB expus, secțiunea 2.3), `usb_module.py` caută periodic printre porturile cunoscute (asociate modulului coș, respectiv modulului braț) și se conectează la oricare este prezent, citindu-i datele prin serial: linii de forma "Weight: ..." pentru coș, respectiv "Mech. Arm: <servo> -> <unghi> deg" pentru braț. Tipul de modul conectat determină automat eticheta afișată în meniul de pe display.

`port_manager.py` este un registru partajat de porturi seriale (claim/release), folosit atât de `usb_module.py`, cât și de `ble_scanner.py`, pentru a garanta că cele două module nu deschid niciodată simultan același port serial fizic.

La conectarea sau deconectarea unui modul, robotul anunță vocal evenimentul ("Cântar conectat", "Braț mecanic conectat" etc.), folosind mecanismul de feedback audio descris în secțiunea 4.10.

4.8. Scanare Coduri de Bare și Cântărire — `barcode_scanner.py`

Funcționalitatea de asistent la cumpărături combină două surse de date: camera web (pentru scanarea codurilor de bare) și load cell-ul modulului coș (pentru cântărire).

Scanarea codurilor de bare se face cu biblioteca `pyzbar`, care procesează cadrele video și detectează coduri EAN-13. La detectarea unui cod nou (cu un mecanism de debounce de câteva secunde, pentru a nu înregistra același produs de mai multe ori), scriptul caută codul într-o bază de date locală de produse și îl adaugă în coș împreună cu prețul; codurile necunoscute sunt marcate ca atare. Sumarul achizițiilor este scris într-un fișier de stare, afișat atât pe display, cât și pe interfața web.

4.9. Siguranță — `tipover_monitor.py`

Un thread separat, independent de navigație sau urmărire, monitorizează continuu înclinarea robotului direct din datele brute ale accelerometrului (fără filtrul Madgwick, fiind suficient un calcul simplu de roll/pitch bazat pe gravitație). Dacă înclinarea depășește un prag configurabil (implicit 45°) pe oricare axă, robotul este considerat răsturnat și emite o alertă vocală ("Am picat! Te rog ridică-mă!"), cu o perioadă de gardă între alerte repetate. Revenirea la o poziție normală este confirmată doar după ce robotul rămâne stabil un scurt interval, pentru a evita declanșări false pe hopuri sau vibrații trecătoare.

4.10. Feedback Audio

Confirmările vocale ale robotului sunt generate cu `espeak`, un motor de sinteză vocală rulat ca proces extern, apelat asincron (fără a bloca bucla principală de control a fiecărui script cât timp se rostește o frază). Vocea este configurată pentru limba română, iar frazele acoperă: testarea motoarelor la pornire, conectarea/deconectarea gamepad-ului, pornirea/oprirea urmăririi prin comandă vocală, conectarea/deconectarea unui modul și alerta de răsturnare. Difuzorul montat în față, sub cameră, redă acest sunet, primit prin mufa jack audio a Raspberry Pi-ului.

4.11. Scripturi de Testare și Diagnostic

Pe lângă scripturile de mai sus, dezvoltarea robotului s-a bazat pe scripturi mici de verificare a componentelor individuale, păstrate pentru depanare rapidă:

Script	Scop
motor_test.py / controller_test.py	Testarea izolată a motoarelor prin PCA9685, respectiv a citirii gamepad-ului PS5.
imu_test.py / calibrate_imu.py	Verificarea orientării fizice a IMU-ului (semnele corecte pe axe) și testarea filtrului Madgwick + heading busolă înainte de integrarea în navigation.py.
esp_serial_test.py	Consolă serială pentru testarea manuală a comunicării cu un ESP32 conectat.
lidar_test.py	Citirea brută a datelor de pe portul serial al LiDAR-ului — a confirmat că modulul montat este defect.
cam_test.py	Prototip inițial de urmărire cu un detector HOG clasic, înainte de trecerea la MobileNet SSD.
display_test.py	Test izolat al inițializării și randării pe display-ul TFT SPI.
mic_test.py	Test izolat al recunoașterii vocale (Vosk) pe microfonul camerei, înainte de integrarea în voice_module.py.
speaker_test.py / audio_speaker.py	Test izolat al feedback-ului audio (espeak).

4.12. Biblioteci și Tehnologii Utilizate

Bibliotecă / Tehnologie	Utilizare în proiect
OpenCV (cv2)	Captură și procesare video, rețeaua DNN MobileNet SSD pentru detecția persoanei.
NumPy	Operații numerice pentru IMU și pentru conversia framebuffer-ului display-ului.
Pillow (PIL)	Randarea interfeței display-ului TFT în memorie, înainte de trimiterea pe SPI.
spidev	Comunicarea la nivel jos cu display-ul TFT prin magistrala SPI a Raspberry Pi.
RPi.GPIO	Control GPIO pentru pinii DC/RST ai display-ului și pinii de enable ai driverelor BTS7960.
Adafruit Blinka (board, busio) + adafruit-circuitpython-pca9685	Interfațarea cu PCA9685 pe I ² C pentru semnalele PWM ale motoarelor.
evdev	Citirea directă a evenimentelor de la gamepad-ul PS5 DualSense (stick-uri, D-pad, butoane).
Flask	Servere web pentru: stream video (buddy_bot.py, barcode_scanner.py), oglindirea display-ului + control de la distanță + editor de traseu (display.py).
pyserial	Comunicare serială cu toate ESP32-urile (module și beacon-uri BLE) și cu LiDAR-ul.
pyzbar	Decodarea codurilor de bare din cadrele video pentru modulul coș.

Bibliotecă / Tehnologie	Utilizare în proiect
qwiiic_icm20948	Driver pentru senzorul IMU ICM-20948 (citire accelerometru/giroscop/magnetometru brut).
ahrs (filtrul Madgwick)	Fuziune senzorială pentru estimarea orientării (roll/pitch) robotului.
SciPy (scipy.spatial.transform.Rotation)	Conversia cuaternionilor produși de filtrul Madgwick în unghiuri Euler.
sounddevice	Capturarea audio de la microfonul camerei web pentru recunoașterea vocală.
Vosk	Recunoaștere vocală offline (bazată pe Kaldi), pentru comenzile vocale ale brațului.
espeak	Sinteză vocală în limba română, pentru feedback-ul audio al robotului.
threading / queue (Python standard)	Arhitectura pe mai multe fire de execuție a tuturor scripturilor.
json (Python standard)	Comunicarea între procese independente prin fișiere de stare/comandă în /tmp.

4.13. Clasificarea Software-ului Conform Criteriilor de Concurs

- **Regim client-server, cu serverul pe robot și clientul de la distanță:** display.py rulează un server Flask (port 8081) care expune display-ul robotului, comenzile echivalente gamepad-ului, editorul de traseu și controlul manual oricărui browser conectat la access point-ul robotului; buddy_bot.py și barcode_scanner.py rulează, similar, servere Flask care transmit fluxul video de la robot către un client de la distanță. Acesta este cel mai ridicat nivel de complexitate atins de software-ul BuddyBot, conform categoriilor din criterii.
- **Program independent, cu interfață pentru utilizator:** gamepad-ul PS5 și display-ul TFT montat pe robot oferă un mod de control și monitorizare complet local, fără a necesita un client de la distanță — util atunci când nu este disponibilă o conexiune la access point-ul robotului.

4.14. Stilul de Programare și Eficiența Codului

- **Separarea captură/inferență:** camera este citită într-un thread propriu, la rata sa naturală, în timp ce inferența MobileNet SSD (cea mai costisitoare operație) rulează într-un thread separat, la o rată fixă (24 Hz) — bucla de control motor nu așteaptă un ciclu de inferență mai lung.
- **Citire serială cu timeout, nu blocantă:** toate conexiunile seriale (ble_scanner.py, usb_module.py) folosesc un timeout scurt la citire, în loc să blocheze indefinit firul de execuție dacă un modul nu trimite date.
- **Citire evenimente gamepad cu select() ne-blocant:** display.py și buddy_bot.py verifică dacă există date noi de la gamepad cu select.select(...) înainte de a citi, în loc de o citire blocantă care ar opri bucla principală în absența unui eveniment.
- **Redimensionare agresivă înainte de inferență:** cadrele sunt reduse la 160×120 pixeli pentru rețeaua neuronală, păstrând o rezoluție mai mare pentru fluxul video afișat utilizatorului.
- **Scriere de stare limitată în rată:** fișierele JSON de stare sunt rescrise de câteva ori pe secundă, nu la fiecare cadru, pentru a cruța cardul SD.
- **Randare condiționată pe display:** fiecare ecran al interfeței TFT se redesenează doar când starea sa relevantă s-a schimbat, nu la fiecare tick al buclei, economisind transferuri SPI inutile.

- **Filtrare exponențială pentru RSSI:** semnalul BLE este stabilizat printr-o medie mobilă exponențială, atenuând zgomotul fără latența unei ferestre fixe.
- **Detecție dinamică a porturilor seriale:** modulele BLE și USB sunt identificate după conținutul liniilor primite, nu după un nume de port fixat în cod, care s-ar putea schimba la o reconectare.
- **Prevenirea conflictelor de port:** `port_manager.py` garantează că două scripturi nu deschid niciodată același port serial simultan.
- **Feedback audio ne-blocant:** apelurile către `espeak` sunt lansate ca subproces, fără a aștepta finalizarea lor, astfel încât o frază lungă nu blochează bucla de control a robotului.

Capitolul V: Design Industrial

5.1. Simplitate și Funcționalitate

Structura care ține cutia de electronice la înălțime este organizată în jurul celor patru picioare motorizate, fiecare găzduind un motor GoBilda și perechea de angrenaje conice care transmite mișcarea la roată — o soluție cu un singur tip de subansamblu repetat de patru ori, în loc de patru variante diferite de picior. Suprafața superioară a cutiei este dedicată în întregime celor două șine liniare pentru module, iar capacul este din Plexiglas, decupat la dimensiune cu un cutter cu fir cald.

Fixarea capacului se face printr-un mecanism cu balama culisantă, asemănător modului de închidere al modulului AMS 2 de la Bambu Lab: capacul culisează și se blochează în poziție închisă, dar poate fi deschis fără scule pentru acces la componentele din interior. Fiecare element mecanic are un rol clar: picioarele asigură garda la sol, șinele asigură montarea modulelor, capacul asigură protecția și accesul rapid — fără elemente decorative sau fără scop funcțional.

5.2. Compatibilitate cu Automatizarea Producției

Toate piesele robotului (structura picioarelor, carcasele modulelor, suporturile) sunt proiectate ca piese imprimate 3D, pornind de la fișiere CAD care pot fi trimise direct la o imprimantă 3D sau, pentru un volum mai mare, folosite ca bază pentru o matriță de injecție, fără modificări majore de proiectare.

Sistemul de șine liniare standardizează modul în care orice modul nou se montează pe platformă: un modul care respectă dimensiunile plăcii de bază și conectorul USB expus este compatibil imediat, fără a fi nevoie să se reproiecteze platforma pentru fiecare modul nou. Această standardizare este ceea ce ar face posibilă producerea în serie a platformei de bază, separat de module, și adăugarea de module noi ca o linie de producție distinctă.

Toate modulele (brațul robotic, coșul de cumpărături, suportul Insta360) au carcase sau suporturi proprii, imprimate 3D, care izolează electric plăcile ESP32 și conexiunile fiecărui modul, reducând riscul de scurtcircuit atunci când modulele sunt manipulate sau depozitate separat de robot.

5.3. Dificultăți Întâmpinate

Cea mai mare provocare a proiectului a fost performanța limitată a Raspberry Pi 3B+, în special cei doar 1 GB de RAM disponibili — o constrângere care a determinat majoritatea deciziilor de optimizare din capitolul de software (redimensionarea cadrelor înainte de inferență, limitarea ratei de inferență, scrierea rară a fișierelor de stare, randarea condiționată a interfeței).

A doua provocare a fost sistemul de alimentare. Într-o versiune anterioară, foloseam cabluri Dupont lungi pentru distribuția tensiunii de 5V către Raspberry Pi, care pierdeau tensiune pe traseu, provocând resetări (undervoltage) ale Raspberry Pi-ului chiar și la sarcini moderate — o problemă greu de diagnosticat inițial, pentru că se manifesta doar intermitent, sub sarcină. Soluția a venit din două direcții: înlocuirea majorității legăturilor lungi cu conectori WAGO și reducerea la minimum (~1 cm) a segmentului de cablu Dupont rămas pe traseul final către pinii Raspberry Pi, completată cu un condensator montat direct pe pinii 5V/GND ai plăcii.

O a treia problemă a fost arderea repetată a driverelor de motor: planul inițial prevedea câte un driver BTS7960 per motor (patru drivere), însă greșeli de cablare au dus la arderea mai multor drivere pe parcursul dezvoltării. Soluția finală, mai simplă, a fost reducerea la doar două drivere, câte unul per parte a robotului, fiecare controlând în paralel cele două motoare de pe partea respectivă.

În fine, LiDAR-ul rotativ montat sub cutia de electronice a ajuns defect din fabricație și transmite doar date incoerente pe portul serial — a rămas montat pe robot ca element de arhitectură, dar nu este folosit în deciziile robotului în stadiul actual.

Bibliografie

Informații:

Raspberry Pi Foundation. (n.d.). Raspberry Pi 3 Model B+ Documentation. Accesat la: <https://www.raspberrypi.com>

OpenCV. (n.d.). OpenCV Documentation - Computer Vision Library. Accesat la: <https://docs.opencv.org>

GoBilda. (n.d.). YellowJacket Planetary Gear Motors & Components. Accesat la: <https://www.gobilda.com>

Espressif Systems. (2024). ESP32-C3 Technical Reference Manual. Accesat la: <https://www.espressif.com>

SparkFun. (n.d.). HX711 Load Cell Amplifier Hookup Guide. Accesat la: <https://learn.sparkfun.com>

SparkFun / TDK InvenSense. (n.d.). ICM-20948 Datasheet — Qwiic 9DoF Hookup Guide. Accesat la: <https://learn.sparkfun.com>

Adafruit. (n.d.). Adafruit PCA9685 16-Channel PWM/Servo Driver Guide. Accesat la: <https://learn.adafruit.com>

Adafruit. (n.d.). Adafruit Blinka — CircuitPython for Raspberry Pi. Accesat la: <https://learn.adafruit.com>

pyserial. (n.d.). Python Serial Port Extension Documentation. Accesat la: <https://pyserial.readthedocs.io>

pyzbar. (n.d.). Python Barcode Scanning Library. Accesat la: <https://pypi.org/project/pyzbar>

evdev Python Library. (n.d.). Linux Input Subsystem Bindings. Accesat la: <https://python-evdev.readthedocs.io>

Flask. (n.d.). Flask Web Framework Documentation. Accesat la: <https://flask.palletsprojects.com>

Vosk. (n.d.). Vosk Offline Speech Recognition Toolkit. Accesat la: <https://alphacephei.com/vosk>

ahrs Python Library. (n.d.). Attitude and Heading Reference Systems — Madgwick Filter. Accesat la: <https://ahrs.readthedocs.io>

SciPy. (n.d.). scipy.spatial.transform.Rotation Documentation. Accesat la: <https://docs.scipy.org>

espeak. (n.d.). eSpeak Text-to-Speech Synthesizer. Accesat la: <http://espeak.sourceforge.net>

BTS7960 Motor Driver Datasheet. (n.d.). Accesat la: <https://www.electroschematics.com>

Pillow (PIL Fork). (n.d.). Python Imaging Library Documentation. Accesat la: <https://pillow.readthedocs.io>

NumPy. (n.d.). NumPy Documentation. Accesat la: <https://numpy.org/doc>

OpenAI. (2025). Asistență tehnică și redactare prin ChatGPT. Accesat la: <https://chat.openai.com>

Anthropic. (2026). Asistență tehnică și redactare a documentației prin Claude. Accesat la: <https://claude.ai>